👑 CODERKING STUDIO

Official Technical Roadmaps & Architecture Guides

ALGORITHMS & HIGH SCALE

Redis Architecture: Why Single-Thread Beats Multi-Thread

Why Redis achieves 100,000+ QPS on a single execution thread using Linux epoll I/O multiplexing and zero context-switch overhead.

1. Memory Speed vs Thread Context Switching

DRAM reads take ~100 nanoseconds. Multi-threaded servers spend thousands of CPU cycles in mutex lock contention and OS thread context switching. Redis avoids locks entirely by running single-threaded.

```
Zero Mutex Locks + Zero Context Switches = 100k+ QPS
```

2. Linux epoll I/O Multiplexing

A single Redis thread monitors 10,000+ simultaneous TCP client sockets using non-blocking event loops with the ae event library:

```
Linux Kernel epoll_wait() → Ready Sockets Queue → Process In-Memory
```

3. Ultra-Fast In-Memory Data Structures

Redis algorithms are tuned for minimum memory indirection and $O(1)$ lookups:

- **Strings**: Simple Dynamic Strings (SDS) with pre-allocated buffer
- **Sorted Sets**: Skiplist + Hash Table ($O(\log N)$ range queries)
- **Hashes**: ZipList for small memory footprint, dict for large scale

4. Production Redis Best Practices

Essential commands for high-performance operations:

```
# Monitor command latency spikes
redis-cli --latency -h 127.0.0.1

# Never use KEYS * in production! Use cursor iteration:
redis-cli --scan --pattern 'user:*' --count 1000
```